Fragen und Antworten: Views und Performance

Views

Welchen Zweck haben Views?

Definition View: ein View ist ein Datenbankobjekt,das das Ergebnis einer per SELECT definierten Abfrage zum Zeitpunkt des Zugriffs auf die View darstellt.

Sie definieren eine feste Abfrage als virtuelle Tabelle. Sie bieten Zugriffskontrolle, Vereinfachung komplexer Abfragen, Wartbarkeit und Konsistenz.

Welche Vorteile bieten Views gegenüber dem direkten Zugriff auf Tabellen?

Zugriffskontrolle: Sensible Daten können ausgeblendet werden. Vereinfachung komplizierter Abfragen in dem sie nur einmal geschrieben werden müssen.

Wartbarkeit: Bei Änderungen in der Datenbank kann die Logik der View angepasst werden ohne das Code anderweitig umgeschrieben werden muss.

Konsistenz: Komplexe Berechnungen können in einer View zusammengefasst werden,dadurch kann sichergestellt werden, dass allen Anwendungen exakt die selbe Logik zugrunde liegt.

Welche Nachteile bzw. Gefahren können entstehen?

Performance(„Black Box“-Effekt"): Die innere Logik ist nicht sichtbar,sodass nicht ersichtlich ist wieviel Rechnleistung nötig ist.

Erschwerte Fehlersuche und Wartung: Man sieht nur den Aufruf der View und musss die zugrundeliegende SQL-Definition separat auslesen.

Kettenabhängigkeiten („Dependency Hell“):Wenn Views auf anderen Views aufbauen können Änderungen andere Views unbrauchbar machen.

Einschränkungen bei Schreiboperationen: In z.B: SQLite sind Views standardmäßig read-only, sobald sie eine gewisse Komplexität überschreiten.

Performance

Liefern die beiden Views tatsächlich dasselbe Ergebnis?

Ja:

```
SELECT *
FROM letzte_bestellung_a
WHERE kunde_id = 42
LIMIT 10000;
;
```

```
--> 42|5212|2026-07-17 06:53:10
```

```
SELECT *
FROM letzte_bestellung_b
WHERE kunde_id = 42
LIMIT 10000;
;
```

```
--> 42|5212|2026-07-17 06:53:10
```

Wie unterscheiden sich die gemessenen Laufzeiten?

View A benötigt bei 1000 Aufrufen rund 93,43 Sekunden (ca. 1:33 Minuten) und bei 10.000 Aufrufen rund 950,11 Sekunden (ca. 15:50 min).

View B benötigt bei 1.000 Aufrufen rund 10,21 Sekunden und bei 10.000 Aufrufen rund 107,19 Sekunden(ca. 1:47 Minuten).

Bei 1.000 Aufrufen benötigt View A um circa 9,15 mal und bei 10.000 Aufrufen circa 8,86 mal länger als View B.

Warum ist es problematisch, dass die interne SQL-Abfrage einer View bei der Verwendung nicht unmittelbar sichtbar ist?

Die Performance ist nicht abzuschätzen, die Fehlersuche ist erschwert und es können versteckte Abhängigkeiten entstehen.

Was zeigt EXPLAIN QUERY PLAN über die beiden Abfragen?

EXPLAIN QUERY PLAN : "Der SQL-Befehl wird verwendet, um eine grobe Beschreibung (High-Level-Beschreibung) der Strategie oder des Plans zu erhalten, den SQLite zur Ausführung einer bestimmten SQL-Abfrage nutzt. Am wichtigsten ist dabei, dass EXPLAIN QUERY PLAN darüber berichtet, auf welche Weise die Abfrage Datenbank-Indizes verwendet."(übersetzter Text aus SQLite Dokumentation vom 06.10.2026)

```
QUERY PLAN
|--SCAN b
`--CORRELATED SCALAR SUBQUERY 3
   `--SEARCH b2
```

SCAN: Es wird ein Full Table Scan der Tabelle b durchgeführt.

CORRELATED SCALAR SUBQUERY 3:

SCALAR: nur ein Wert wird zurückgegeben.

CORRELATED SUBQUERY: die Werte ändern sich in Abhängigkeit der aktuellen Zeile der äußeren Abfrage.

SEARCH: sagt das nur ein Subset von Tabellen Zeilen abgesucht wird.

```
 QUERY PLAN
|--CO-ROUTINE letzte
|  `--SCAN bestellung
|--SCAN b
|--BLOOM FILTER ON letzte (kunde_id=?)
`--SEARCH letzte USING AUTOMATIC PARTIAL COVERING INDEX (kunde_id=?)
```

CO-ROUTINE: ist eine Unterabfrage, deren Daten während der Ausführung Zeile für Zeile on-the-fly und ohne vorherige Zwischenspeicherung in eine temporäre Tabelle an die Hauptabfrage geliefert werden.

SCAN: Es wird ein Full Table Scan der Tabelle bestellung dann der Tabelle b durchgeführt.

BLOOM FILTER: " eine probabilistische* Datenstruktur, mit deren Hilfe sehr schnell festgestellt werden kann, welche Daten in einem Datenstrom schon einmal vorgekommen sind und welche erstmals auftreten"(aus https://de.wikipedia.org/w/index.php?title=Bloomfilter&oldid=260174336) schließt nicht vorhandene Kunden-IDs aus.

(kunde_id=?): Suchbedingung mit ? als Platzhalter für variablen Wert.

SEARCH: sagt das nur ein Subset von Tabellen Zeilen abgesucht wird in diesem Fall letzte.

AUTOMATIC PARTIAL COVERING INDEX: ein vom Optimizer(siehe unten) automatisch erstellter temporärer Index der Partial Index(siehe unten) und Covering Index(siehe unten) vereint.

Optimizer: Abfrageoptimierer"...ist Teil eines Datenbankmanagementsystems (DBMS), der versucht, für eine Datenbankabfrage einen optimalen Auswertungsplan zu berechnen."(aus https://de.wikipedia.org/w/index.php?title=Abfrageoptimierer&oldid=267843644)

PARTIAL INDEX: "Ein partieller Index ist ein Index über eine Teilmenge der Zeilen einer Tabelle.

Bei gewöhnlichen Indizes gibt es für jede Zeile in der Tabelle exakt einen Eintrag im Index. Bei partiellen Indizes besitzen nur einige Zeilen der Tabelle entsprechende Indexeinträge. Beispielsweise könnte ein partieller Index Einträge auslassen, bei denen die indizierte Spalte NULL ist. Sinnvoll eingesetzt, können

partielle Indizes zu kleineren Datenbankdateien sowie zu Verbesserungen bei der Abfrage- und der Schreibleistung führen." (übersetzter Text aus SQLite Dokumentation vom 06.10.2026)

COVERING INDEX: Ein Covering Index ist ein Datenbank-Index, der alle Spalten enthält, die eine bestimmte Abfrage benötigt. Dadurch muss die Datenbank nach dem Durchsuchen des Index nicht mehr auf die eigentliche Datentabelle zugreifen, um fehlende Werte nachzuladen. Das spart pro Zeile einen kompletten Suchschritt (eine binäre Suche) und kann Abfragen erheblich beschleunigen.

Welche Unterschiede in der Verarbeitung erklären die gemessenen Laufzeiten?

Variante A sucht Zeile für Zeile während Variante B einen durch den Optimizer (Abfrageoptimierer)erstellten Index benutzt und damit deutlich schneller ist.

Conclusio

Eine View vereinfacht zwar die Handhabung und Lesbarkeit im Code, aber wie aus den Messungen hervorgeht(bei 1.000 Aufrufen ist View A um den Faktor 9,15 und bei 10.000 Aufrufen um 8,86 langsamer), jedoch nicht die Komplexität der zugrunde liegenden Abfrage noch deren Perfomanz.

---

*auf Wahrscheinlichkeiten beruhend

Gesamtarbeitszeit : 12 Stunden 20 Minuten